

Tugas Kelompok

Nama Kelompok :

- Muhammad Gibrail Brian Tou (60525025)
- Mohammad Dexan Oktan Ramadhan (60525024)
- Adiana Muchamad Rizky (60525037)
- Agridan Nasywan (60525032)
- Reva Azya Fahreza (60525033)
- Isan Ainul Bagja

Mata Kuliah : Algoritma Dan Pemograman

Dosen Pengampu : Patah Herwanto, S.T., M.KOM

Sistem Manajemen Inventaris Toko (Multi-Kategori)

Untuk setiap kasus, Buat :

1. Desain Struktur Data

o Tabel/diagram berisi field dan tipe datanya.

2. Flowchart Global

o Alur utama menu aplikasi CRUD.

3. Pseudocode / Algoritma

o Minimal untuk operasi CRUD utama.

4. Implementasi Kode (Python)

o Menggunakan list / list of list.

5. Laporan Singkat

o Penjelasan fitur yang berhasil dibuat

o Kendala dan solusi.

1. Sistem Manajemen Inventaris Coffee Shop

Deskripsi Singkat:

Sebuah minimarket ingin mengelola data stok barang secara digital.

Struktur Data Utama (Produk):

- kode_barang
- nama_barang
- kategori (misal: Makanan, Minuman, ATK, dll.)
- stok
- harga_satuan
- status (Aktif / Tidak Aktif)

Fitur Wajib:

1. CREATE: Tambah barang baru, cek duplikasi kode_barang.
2. READ: Tampilkan daftar barang dalam bentuk tabel (bisa difilter per kategori).
3. UPDATE: Ubah nama, kategori, stok, atau harga suatu barang.
4. DELETE (Soft Delete): Ubah status menjadi "Tidak Aktif" alih-alih benar-benar menghapus.
5. SORTING:
 - o Urutkan berdasarkan nama_barang (A–Z)
 - o Urutkan berdasarkan stok (rendah → tinggi)
6. SEARCHING:
 - o Cari barang berdasarkan kode_barang
 - o Cari berdasarkan kata kunci pada nama_barang
7. Laporan:
 - o Total nilai stok ($\sum \text{stok} \times \text{harga_satuan}$ per barang)
 - o Daftar barang yang stoknya < batas minimum (misal < 5)
8. Fitur restock: tambah stok dengan catatan histori restocksederhana (tanggal + jumlah).

Sistem Manajemen Inventaris Coffee Shop

1. Desain Struktur Data

Struktur data utama menggunakan list of lists untuk menyimpan data produk. Setiap produk direpresentasikan sebagai list dengan indeks sebagai berikut:

- Indeks 0: kode_barang (string)
- Indeks 1: nama_barang (string)
- Indeks 2: kategori (string)
- Indeks 3: stok (integer)
- Indeks 4: harga_satuan (float)
- Indeks 5: status (string: "Aktif" atau "Tidak Aktif")
- Indeks 6: histori_restock (list of lists: [[tanggal (string), jumlah (integer)], ...])

Tabel representasi:

Indeks	Field	Tipe Data	Deskripsi
0	kode_barang	string	Kode unik barang (misal: "CB001")
1	nama_barang	string	Nama barang (misal: "Kopi Hitam")
2	kategori	string	Kategori (misal: "Minuman", "Makanan")
3	stok	integer	Jumlah stok (misal: 10)
4	harga_satuan	float	Harga per unit (misal: 15000.0)
5	status	string	Status ("Aktif" atau "Tidak Aktif")
6	histori_restock	list of lists	Histori restock: [[tanggal, jumlah], ...]

Contoh data:

[["CB001", "Kopi Hitam", "Minuman", 10, 15000.0, "Aktif", [["2023-10-01", 5], ["2023-10-05", 3]]]

2. Flowchart Global

Flowchart alur utama menu aplikasi CRUD (dalam bentuk deskripsi teks, karena ini teks-based):

1. Start: Jalankan program.
2. Tampilkan Menu Utama:
 1. Tambah Barang (CREATE)
 2. Tampilkan Barang (READ) - dengan opsi filter kategori
 3. Update Barang (UPDATE)
 4. Hapus Barang (Soft Delete)
 5. Sorting Barang (nama A-Z atau stok rendah-tinggi)
 6. Searching Barang (kode atau nama)
 7. Laporan (total nilai stok, barang stok < 5)
 8. Restock Barang
 9. Keluar
3. Input Pilihan: User memilih menu.
4. Proses Berdasarkan Pilihan:
 - Jika CREATE: Input data baru, cek duplikasi kode, tambahkan ke list.
 - Jika READ: Tampilkan tabel, opsi filter kategori.
 - Jika UPDATE: Cari berdasarkan kode, update field.
 - Jika DELETE: Cari berdasarkan kode, ubah status ke "Tidak Aktif".
 - Jika SORTING: Pilih jenis sort, tampilkan hasil.
 - Jika SEARCHING: Input kata kunci, tampilkan hasil.
 - Jika LAPORAN: Hitung total nilai stok, tampilkan barang stok < 5.
 - Jika RESTOCK: Cari berdasarkan kode, tambah stok, catat histori.
 - Jika KELUAR: End.
- 5.
6. Loop Kembali ke Menu: Setelah proses, kembali ke menu utama kecuali keluar.
7. End: Program selesai.

3. Pseudocode / Algoritma

Minimal untuk operasi CRUD utama (CREATE, READ, UPDATE, DELETE). Menggunakan list of lists sebagai inventory.

inventory = [] # List of lists untuk menyimpan produk

FUNCTION CREATE(kode, nama, kategori, stok, harga, status="Aktif", histori=[]):

FOR each item IN inventory:

IF item[0] == kode:

RETURN "Duplikasi kode_barang"

```

NEW_ITEM = [kode, nama, kategori, stok, harga, status, histori]

APPEND NEW_ITEM to inventory

RETURN "Berhasil ditambah"

FUNCTION READ(filter_kategori=None):

    IF filter_kategori IS NOT None:

        filtered = [item FOR item IN inventory IF item[2] == filter_kategori AND item[5] ==
"Aktif"]

    ELSE:

        filtered = [item FOR item IN inventory IF item[5] == "Aktif"]

    PRINT table of filtered

FUNCTION UPDATE(kode, new_nama=None, new_kategori=None, new_stok=None,
new_harga=None):

    FOR each item IN inventory:

        IF item[0] == kode:

            IF new_nama: item[1] = new_nama

            IF new_kategori: item[2] = new_kategori

            IF new_stok: item[3] = new_stok

            IF new_harga: item[4] = new_harga

            RETURN "Berhasil diupdate"

    RETURN "Barang tidak ditemukan"

FUNCTION DELETE(kode):

    FOR each item IN inventory:

        IF item[0] == kode:

            item[5] = "Tidak Aktif"

            RETURN "Berhasil dihapus (soft delete)"

    RETURN "Barang tidak ditemukan"

```

4. Implementasi Kode (Python)

Menggunakan list of lists. Kode lengkap dengan menu interaktif.

```
import datetime

# Struktur data: list of lists

inventory = []

def create_item(kode, nama, kategori, stok, harga, status="Aktif", histori=None):

    if histori is None:

        histori = []

    for item in inventory:

        if item[0] == kode:

            return "Duplikasi kode_barang"

    new_item = [kode, nama, kategori, stok, harga, status, histori]

    inventory.append(new_item)

    return "Berhasil ditambah"

def read_items(filter_kategori=None):

    filtered = []

    for item in inventory:

        if item[5] == "Aktif" and (filter_kategori is None or item[2] == filter_kategori):

            filtered.append(item)

    if not filtered:

        print("Tidak ada barang yang sesuai.")

        return

    print("Daftar Barang:")

    print(f'{"Kode":<10} {"Nama":<20} {"Kategori":<15} {"Stok":<5} {"Harga":<10} {"Status":<10}')

    for item in filtered:

        print(f'{"item[0]":<10} {"item[1]":<20} {"item[2]":<15} {"item[3]":<5} {"item[4]":<10} {"item[5]":<10}')

def update_item(kode, new_nama=None, new_kategori=None, new_stok=None,
new_harga=None):
```

```

for item in inventory:

    if item[0] == kode:

        if new_nama: item[1] = new_nama

        if new_kategori: item[2] = new_kategori

        if new_stok: item[3] = new_stok

        if new_harga: item[4] = new_harga

        return "Berhasil diupdate"

    return "Barang tidak ditemukan"

def delete_item(kode):

    for item in inventory:

        if item[0] == kode:

            item[5] = "Tidak Aktif"

            return "Berhasil dihapus (soft delete)"

    return "Barang tidak ditemukan"

def sort_items(by="nama"):

    if by == "nama":

        inventory.sort(key=lambda x: x[1])

    elif by == "stok":

        inventory.sort(key=lambda x: x[3])

    print("Barang telah diurutkan.")

def search_items(keyword, by="kode"):

    results = []

    for item in inventory:

        if item[5] == "Aktif":

            if by == "kode" and keyword.lower() in item[0].lower():

                results.append(item)

```

```

        elif by == "nama" and keyword.lower() in item[1].lower():

            results.append(item)

if not results:

    print("Tidak ada hasil pencarian.")

    return

print("Hasil Pencarian:")

print(f'{"Kode":<10} {"Nama":<20} {"Kategori":<15} {"Stok":<5} {"Harga":<10} {"Status":<10}')

for item in results:

    print(f'{"item[0]:<10} {"item[1]:<20} {"item[2]:<15} {"item[3]:<5} {"item[4]:<10} {"item[5]:<10}')

def generate_report():

    total_nilai = 0

    low_stock = []

    for item in inventory:

        if item[5] == "Aktif":

            total_nilai += item[3] * item[4]

            if item[3] < 5:

                low_stock.append(item)

    print(f'Total Nilai Stok: {total_nilai}')

    if low_stock:

        print("Barang dengan stok < 5:")

        for item in low_stock:

            print(f'{"item[0]} - {"item[1]}: {"item[3]}')

    else:

        print("Tidak ada barang dengan stok < 5.")

def restock_item(kode, jumlah):

    for item in inventory:

```



```

    if item[0] == kode and item[5] == "Aktif":

        item[3] += jumlah

        tanggal = datetime.date.today().isoformat()

        item[6].append([tanggal, jumlah])

        return "Berhasil direstock"

    return "Barang tidak ditemukan atau tidak aktif"

# Menu utama

while True:

    print("\nMenu Sistem Manajemen Inventaris Coffee Shop:")

    print("1. Tambah Barang")

    print("2. Tampilkan Barang")

    print("3. Update Barang")

    print("4. Hapus Barang")

    print("5. Sorting Barang")

    print("6. Searching Barang")

    print("7. Laporan")

    print("8. Restock Barang")

    print("9. Keluar")

    pilihan = input("Pilih menu: ")

    if pilihan == "1":

        kode = input("Kode Barang: ")

        nama = input("Nama Barang: ")

        kategori = input("Kategori: ")

        stok = int(input("Stok: "))

        harga = float(input("Harga Satuan: "))

```

```

    print(create_item(kode, nama, kategori, stok, harga))

elif pilihan == "2":

    filter_kat = input("Filter kategori (kosongkan jika tidak): ")

    read_items(filter_kat if filter_kat else None)

elif pilihan == "3":

    kode = input("Kode Barang: ")

    new_nama = input("Nama Baru (kosongkan jika tidak): ") or None

    new_kat = input("Kategori Baru (kosongkan jika tidak): ") or None

    new_stok = input("Stok Baru (kosongkan jika tidak): ")

    new_stok = int(new_stok) if new_stok else None

    new_harga = input("Harga Baru (kosongkan jika tidak): ")

    new_harga = float(new_harga) if new_harga else None

    print(update_item(kode, new_nama, new_kat, new_stok, new_harga))

elif pilihan == "4":

    kode = input("Kode Barang: ")

    print(delete_item(kode))

elif pilihan == "5":

    by = input("Sort by (nama/stok): ")

    sort_items(by)

elif pilihan == "6":

    keyword = input("Kata kunci: ")

    by = input("Cari by (kode/nama): ")

    search_items(keyword, by)

elif pilihan == "7":

    generate_report()

elif pilihan == "8":

```

```

kode = input("Kode Barang: ")

jumlah = int(input("Jumlah Restock: "))

print(restock_item(kode, jumlah))

elif pilihan == "9":

    break

else:

    print("Pilihan tidak valid.")

```

5. Laporan Singkat

Penjelasan Fitur yang Berhasil Dibuat

- CRUD Lengkap: CREATE (tambah barang dengan cek duplikasi), READ (tampilkan tabel dengan filter kategori), UPDATE (ubah field tertentu), DELETE (soft delete dengan ubah status).
- Sorting: Urutkan berdasarkan nama (A-Z) atau stok (rendah-tinggi).
- Searching: Cari berdasarkan kode_barang atau kata kunci di nama_barang.
- Laporan: Hitung total nilai stok ($\sum \text{stok} \times \text{harga_satuan}$), tampilkan daftar barang stok < 5 .
- Restock: Tambah stok dengan catatan histori (tanggal + jumlah) menggunakan datetime.
- Menu Interaktif: Semua fitur diintegrasikan dalam loop menu.

Kendala dan Solusi

- Kendala: Mengelola histori restock sebagai list of lists dalam list utama; input user untuk update (banyak field opsional) bisa rumit.
 - Solusi: Gunakan list of lists untuk histori, dan dalam update gunakan input kosong untuk skip field. Tambahkan validasi input (misal int/float) untuk menghindari error.
- Kendala: Sorting dan searching case-insensitive untuk nama/kode.
 - Solusi: Gunakan .lower() dalam pencarian untuk case-insensitive.
- Kendala: Tidak ada persistensi data (data hilang saat program restart).
 - Solusi: Dalam scope ini, tidak diperlukan; bisa ditambahkan file I/O jika perlu, tapi sesuai instruksi, cukup list in-memory.